PROTODUINO · DESIGN DOCUMENT

A streaming ELF loader for AVR

Functional and technical design of a compile-time configurable ELF32 loader: bootloader core, flash service and loadable processes. With measurements from a working prototype and fixes for current protoduino caveats.

Draft 0.1 · 12 November 2026

jklarenbeek/protoduino · branch ccr-8617b1c3-gjnqlx

84 B

of loader state on an ATmega328P; no heap, no seeking

1,640 B

serial ELF bootloader for the ATmega328P, as measured

2,036 B

with the run-time module loader: it fills the 2 KB boot section

21 of 21

simulated scenarios behaved as designed, including module loading on Optiboot

What is real in this document

- A prototype of the loader core and of a serial bootloader built on it was written and run for this document. Every figure marked MEASURED comes from it: avr-gcc 7.3.0, and simavr at the commit that protosim pins.
- Nothing has run on hardware. Power loss during an update was not simulated, and simavr's flash model is more forgiving than a chip (8.1).
- Run-time module loading was exercised with a stand-in kernel, not with protoduino's scheduler (6.8). Relocation (4.7) and the ATtiny path (6.6) are designs. The ATtiny build compiles; it has not executed anywhere.
- No protoduino, protosim or simavr source file was changed. The fixes in section 7 and section 8 are proposals. The prototype sits beside this document in `docs/elfloader/prototype/`, outside `src/`.

MEASURED built and run for this document READ established by reading source or tool output PROPOSED designed, not built BUILT compiles, not executed

Contents

1 Summary
2 Goals, non-goals and target parts
3 What earlier loaders got wrong
4 The ideas the design rests on
5 Functional specification
6 Technical design
7 Fixes for current protoduino caveats
8 Simulator findings: simavr and protosim
9 Verification
10 Roadmap, risks and open questions
A ELF fields the loader reads
B Reproducing the measurements · Sources

1 Summary

The loader takes the file the linker writes, the `.elf`, and treats it as a stream. The first 52 bytes say what the file is. The next 32 bytes per program header say which bytes belong where. Everything after that is delivered to flash in the order it arrives. The loader never seeks backwards and never allocates, and all of its state fits in one 84-byte structure. When the last loadable byte has been delivered it is done; the symbol tables and debug information that make up most of the file never have to be sent.

That one core is used in parts, chosen at compile time:

- **As a bootloader.** Core, a polled serial port and XMODEM-CRC in the boot section. Any terminal program can send the unmodified `.elf`. An image record written last decides whether the application may start.
- **As a boot ROM for the running system.** The same boot section exports a small table: an entry that programs one page and, when it fits the section, the whole load session. A kernel can then have a module loaded while it runs for under a kilobyte of its own code.
- **Inside the kernel.** Where the bootloader is Optiboot, or a protoduino loader built without the module loader, the kernel links the same session code itself and writes pages through whatever the bootloader offers.

Module support has three levels, also chosen at compile time: none; start the modules that are already in flash; or load modules at run time as well. The build fills the boot section that the fuses reserve with as many features as fit, in a set order of priority (6.9).

Modules fit protoduino in particular. Protothread processes keep their state in a struct the kernel hands them, so a module needs no static RAM, and the hardest part of dynamic loading on an AVR, relocating data addresses, does not arise.

Result	Value	Status
Loader state, ATmega328P build	84 bytes, of which 52 are header scratch	MEASURED
Serial ELF bootloader, ATmega328P	1,640 B flash, 350 B RAM. With the run-time module loader 2,036 B, which fills the 2 KB boot section to within 12 bytes. With every option 2,286 B, which needs the 4 KB section.	MEASURED
Serial ELF bootloader, ATmega2560	2,182 B. The 2 KB section holds a 2,024 B build with the page-write service; the module loader (2,524 B) and every option (2,832 B) need the 4 KB section.	MEASURED
ATtiny85 build with bit-banged serial	1,700 B flash, 286 B RAM	BUILT
Flash written by the loader against simavr's own ELF reader	Identical on both parts, including a 106 KB image that reaches far flash	MEASURED
A file for the wrong core family, offered to a device with a working application	Refused with a reason code; nothing erased; the application starts again	MEASURED
A module sent to a running kernel, loaded by the boot section on the kernel's behalf	Written into its slot, run, recorded, and started again after a reset	MEASURED
The same with a stock Optiboot 8.3 as the bootloader	Works: the kernel carries the loader and writes through Optiboot's `do_spm`	MEASURED
A module that declares static RAM	Fails at link time with a clear message	MEASURED

The size is the weak point, and it should be stated plainly. Optiboot fits in under 512 bytes; this loader is 1,640 bytes, in plain C with an old compiler, and it does not fit a 1 KB boot section. Section 6.10 shows where the bytes go and what could shrink them. Reaching 1 KB is a goal, not a result.

2 Goals, non-goals and target parts

Goals

- **Accept the linker's own output.** No hex file, no binary image and no packaging step for firmware.
- **One core, used in parts.** From a bootloader to a loader inside the kernel, with features that cost nothing when switched off.
- **Fill the boot section.** Whatever block the fuses reserve is used for as many features as fit.
- **Refuse before erasing.** The wrong file must be rejected while the installed image is still intact.
- **Always recoverable.** On parts with a hardware boot section, no failed update may leave a device that cannot be reached over the same serial line.
- **Modules are processes, loadable at run time.** Wherever the part can carry it and unless it is configured out. Three levels: none, modules already in flash, run-time loading.
- **Not tied to our bootloader.** A protoduino kernel can load modules on top of Optiboot as well.

Non-goals

- **Linking on the device.** Relocatable `.o` files, symbol tables and name lookups stay on the host.
- **Authenticated updates in the first version.** Integrity is checked; origin is not. A place for a signature is reserved (6.8).
- **C++ runtime features in modules.** No exceptions, RTTI or static constructors.
- **Parts without room for a page buffer.** The ATtiny4/5/9/10 have 32 bytes of RAM.
- **UPDI-era parts in this document.** tinyAVR 0/1/2 and AVR Dx program flash through NVMCTRL. They need another flash back-end, which the layering allows but which is not designed here.

Part	Flash	RAM	Page	Boot section	Bootloader	Run-time modules
ATmega328P	32 KB	2 KB	128 B	0.5 to 4 KB	MEASURED	Yes. The module loader fits the 2 KB section. MEASURED
ATmega2560	256 KB	8 KB	256 B	1 to 8 KB	MEASURED	Yes. The module loader needs the 4 KB section. BUILT
ATmega1284P	128 KB	16 KB	256 B	1 to 8 KB	BUILT	As the ATmega2560. BUILT
ATtiny85	8 KB	512 B	64 B	none	BUILT	Off by default: the load workspace alone is 286 of its 512 bytes of RAM.
ATtiny25 / 45	2 / 4 KB	128 / 256 B	32 / 64 B	none	Needs a small-frame transport PROPOSED	No. Excluded at compile time.
ATtiny4/5/9/10	≤ 1 KB	32 B	–	none	Out of scope	–

Flash end, RAM range, page size and core family were read from the avr-libc device headers. Boot-section sizes are from the datasheets and were not re-checked here. The request named an “ATtiny8”; no such part exists, so this document takes the ATtiny25/45/85 family as the small target, with the ATtiny85 as reference.

3 What earlier loaders got wrong

Five existing designs were examined, four of them in source. Each got something right that this design keeps, and each left something that it changes.

Loader	What it does, and where it falls short	What this design does instead
Contiki ELF loader 2005 to 2017 READ	Loads relocatable `.o` files and links them on the device. It makes 14 seek-and-read calls, recognises ten sections by comparing name strings, and resolves every import with `strcmp` against a table of names compiled into the kernel. So the whole file must already sit in a seekable store, and the kernel carries its symbol names in flash. It checks the first seven identification bytes and never the machine type. Contiki later added a pre-linked format (CLE), and Contiki-NG deleted the loader in 2017, 4,203 lines in one commit.	No linking on the device. The file is consumed as it arrives. Machine and core family are checked from the header. What Contiki got right is kept: a module is a set of processes, and architecture code is separate from the core.
Optiboot with avrdude	Under 512 bytes, which is the bar for size. But the device receives anonymous pages: it has no idea what the image is, keeps no record of whether it is complete, and starts whatever is there after a timeout. The host has to turn the ELF into a hex file and speak a programmer protocol. Version 8 added `do_spm()` so an application can write flash.	The device reads the ELF header itself and keeps an image record. The `do_spm()` idea is kept and widened into a service table (6.7). Optiboot also stays a supported choice: a protoduino kernel loads modules on top of it (6.8).
urboot, micronucleus parts without a boot section	They patch the application's reset vector so that reset reaches the loader, and rely on erased flash running through to it. The known hazard, in urboot's own words: if the first page is erased but not rewritten, the part is bricked until it meets an ISP programmer.	Erase from the top down with page 0 last, and give the loader a landing pad. There is then no instant at which a reset fails to reach the loader (6.6). PROPOSED The run-through and the landing pad were checked in simulation (T13).
simple-avr-bootloader m3y54m; a tutorial by its own description READ	Sends a raw binary with a 16-bit length. Frames carry an 8-bit additive checksum and no sequence number, and the uploader retries on timeout, so a lost acknowledgement makes the same chunk land twice. The update flag sits 8 bytes below the top of RAM, where the stack starts. The loader runs its serial port from interrupts and has no check that the application is whole.	XMODEM-CRC: 16-bit CRC and block numbers, so a repeat is acknowledged and dropped. A polled serial port with no vectors. An image record. Its good idea is kept: the application asks for an update, then resets.
Host ELF libraries libelf, ELFIO and others	They assume random access or the whole file in memory, and they allocate. None is built to be fed.	Used only as oracles in tests.
Zephyr LLEXT a current reference	Still links on the device, but can replace symbol-name strings with hashes to save space. That is the right direction, and still far too much code for an 8-bit part.	One step further: no symbol table on the device at all. A module is linked on the host against one firmware build and carries that build's identity (6.8).
protoduino `docs/loader.md` READ	Describes modules that are stored in RAM and run from there, which an AVR cannot do: code executes only from flash. It compiles with `-fpic`, which avr-gcc reports as unsupported. It refers to functions that are not in `src/`. Its file header has no magic number, version or checksum.	Superseded by this document.

An earlier note of mine questioned the order in which simple-avr-bootloader fills, erases and writes a page. That was wrong: the datasheet allows the page buffer to be filled either before or after the erase.

4 The ideas the design rests on

4.1 The ELF file is the wire format

An executable from avr-gcc is laid out in the order a loader needs it. The ELF header comes first, the program headers follow at offset 52, the loadable bytes come next, and everything a loader has no use for comes last. A reader that only moves forward can therefore do the whole job, and can stop early.

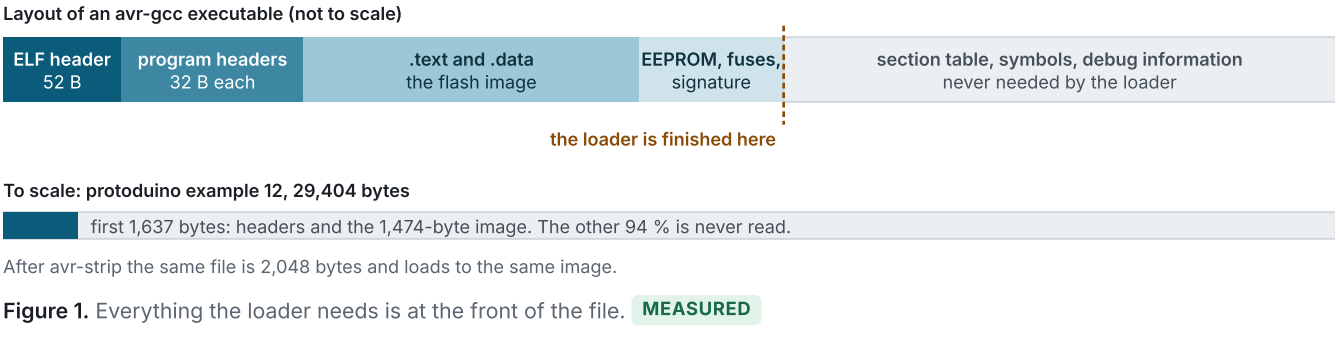


Figure 1. Everything the loader needs is at the front of the file. MEASURED

File	File size	Loader finished within	Result against `avr-objcopy`
Test program, ATmega328P, with EEPROM data, fuses and signature	12,860 B	660 B	Flash and EEPROM identical
protoduino example 12 (UART echo), ATmega328P	29,404 B	1,637 B	Identical, also when fed one byte at a time
106 KB test image, ATmega2560	117,616 B	106,986 B	Identical

The consequence for the user is that no tool has to be installed to flash a board. With XMODEM as the transport (6.4), a terminal program that every platform already has can send the file the build produced.

4.2 One core, used in parts

The core is a state machine with no input or output of its own. Bytes are pushed in from whatever carries them; bytes that belong in a memory are handed to a sink. Around it sits one session, the code that receives a file and drives the core, written so that all of its state is in a workspace the caller owns. That makes the session usable from three places: the bootloader at reset, the boot section called by the running kernel, and a kernel that links it in.

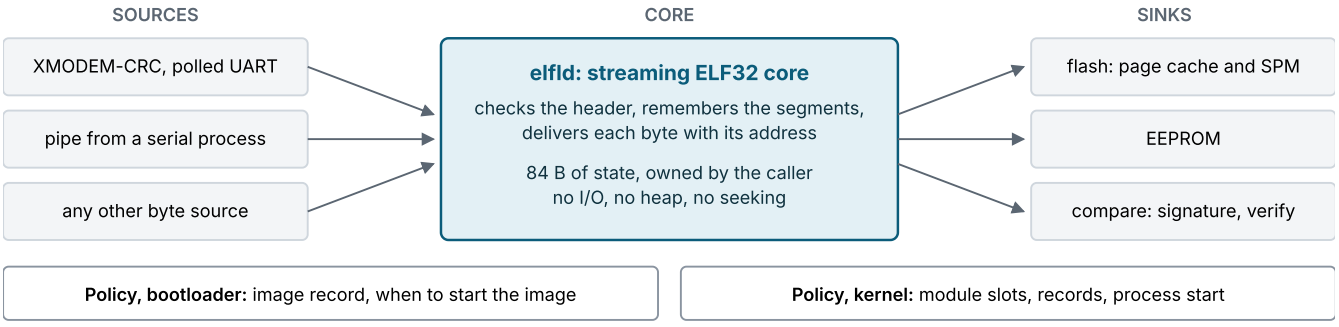


Figure 2. The pieces. The top row of each column is what the prototype contains.

Which parts end up where depends on two choices: what sits in the boot section, and how much module support the kernel is built with.

In the boot section	Kernel built for	Who loads a module at run time	Tested
protoduino loader with the module loader	run-time loading, ROM	The boot section, called by the kernel	M1
protoduino loader with the page-write service	run-time loading, own loader	The kernel; pages go through the service	M3
Optiboot 8 or later	run-time loading, own loader	The kernel; pages go through `do_spm`	M2
anything	modules already in flash	Nobody. Modules are programmed with the firmware and only started.	M4
anything	no modules	–	M8

4.3 Address spaces select the sink

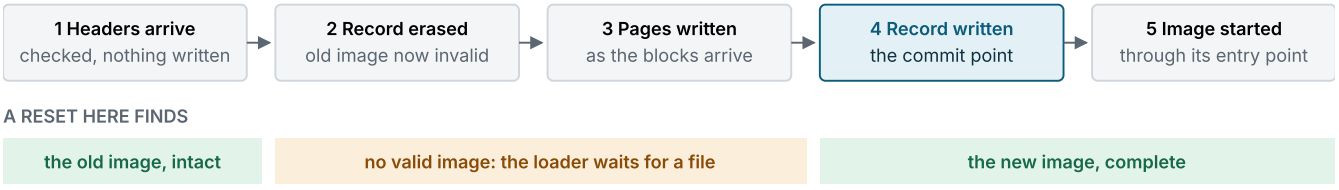
avr-gcc gives every memory of the chip its own range in the ELF address space, so the top byte of a segment's load address says where its bytes belong. The loader routes on that byte. It uses the load address (`p_paddr`), not the run address: the initial values of `.data` are stored in flash right after the code, and the program's own start-up copies them to RAM.

Load address	Memory	What the loader does	Option
`0x00....` upward	Flash	Writes, inside the permitted window only	always
`0x80....`	RAM	Nothing to write. `.bss` has no file bytes. The declared extent can be recorded.	`ELFLD_FEAT_RAM_EXTENT`
`0x81....`	EEPROM	Writes the EEPROM image	`ELFLD_FEAT_EEPROM`
`0x82....`, `0x83....`	Fuses, lock bits	Ignores; software cannot write them	–
`0x84....`	Signature	Compares with the chip's own signature bytes	`ELFLD_FEAT_SIGNATURE`

Two checks come free with the format. The header's flags name the core family the file was built for (`avr:5` for an ATmega328P, `avr:6` for an ATmega2560, `avr:25` for an ATtiny85), and avr-gcc defines the same number as `__AVR_ARCH__` when the loader is compiled. One comparison rejects a file for the wrong family from its first 52 bytes. The signature check is finer, down to the exact device, but it comes late, because avr-gcc places the signature after the flash data (see FR-9).

4.4 Refuse before erasing, commit last

An update is a transaction with one commit point. Nothing in flash changes until every program header has been accepted. The installed image is declared invalid before its first byte changes, and becomes valid again only when a small record is written after the last byte. A reset at any moment therefore finds either the old image, or no valid image and a waiting loader, or the new image.



A refusal in step 1 leaves the device exactly as it was. The first prototype got this wrong; test T7 found it.

Figure 3. The update as a transaction. Steps and refusals were exercised in simulation; a power cut was not. MEASURED

4.5 The boot section as a small ROM

On a megaAVR the SPM instruction works only from the boot section, so an application cannot program flash by itself, and a run-time loader in the application cannot exist without help. The boot section therefore exports a table at a fixed place at the very end of flash. Its first service programs one page from the caller's buffer, as Optiboot's `do_spm()` does. Its second goes further: it runs the whole load session, the same code that loads firmware at reset, on a workspace the caller brings. The kernel asks for a module to be loaded into a range of flash and gets back a result and the module's entry address. The boot section is paid for anyway; this makes it do the kernel's loading too (6.7). MEASURED

4.6 A module is a process type

protoduino already has the right descriptor. `process_type_t` is a small record in flash holding a name, a thread function, an instance size and a priority, and `process_new(type, storage)` turns it into a running process in memory the caller supplies. A loadable module is exactly that record plus its code, linked into a flash slot. Loading a module means writing its bytes and calling the two functions every built-in process already uses. A stand-in kernel showed the mechanics end to end (6.8); joining them to the real scheduler is the next step.

Protothreads are stackless, and a protoduino process keeps its variables in its instance struct. A module written that way has no `.data` and no `.bss`, so there is nothing in RAM to place or relocate: the kernel hands the thread a pointer, and the module works wherever that points. The rule is enforced at link time rather than left to discipline (6.8).

4.7 Relocation without a relocator

The first module scheme links each module for one slot, which needs no code on the device. To let one file load into any slot, addresses inside the module must be adjusted, and the usual way needs a relocation engine. There is a cheaper way. If slots are aligned to 512 bytes, moving a module changes only the high byte of every byte address and of every word address; low bytes never change and no carry crosses between them. A fix-up then is one position in the stream whose high byte gets a constant added. The host sorts those positions by offset and puts the list ahead of the code, and the loader applies them as the bytes go by, still in one pass. On parts with absolute call instructions, calls from the module into the kernel need no fix-up at all. PROPOSED

4.8 The same core on the host, and the simulator as oracle

The core is plain C with no AVR dependency, so it compiles unchanged for the host. That gives three things: unit tests and fuzzing at desktop speed; a host tool that can say exactly what a device will accept, because it runs the device's code; and, compiled to WebAssembly, the same check inside a Node command-line tool or protosim's web front end. For end-to-end tests, the simulator already contains a trusted ELF reader. Flashing a file through the loader inside simavr and comparing the result with what simavr's own reader produces for the same file tests the loader against an independent implementation. Section 9 reports that comparison.

5 Functional specification

5.1 Use cases

ID	Use case	How it goes	Status
UC1	Flash firmware over a serial line	Reset the board. The loader asks for a file by sending `C`. Send the `.elf` with XMODEM from any terminal program. The loader writes it, stores the record and starts it.	MEASURED
UC2	Recover a board whose application is broken or half-written	The record is missing or damaged, so after reset the loader keeps asking for a file. No programmer is needed on parts with a boot section.	MEASURED

ID	Use case	How it goes	Status
UC3	The application asks for an update	It calls `elfboot_request()`, which sets a flag that survives reset and resets through the watchdog. The loader sees the flag and waits for a file.	PROPOSED
UC4	Check what is installed	Send the same file in verify mode. Bytes are compared with flash instead of written.	PROPOSED
UC5	Ask the device what it is	Send `?` before a transfer. The loader answers with part, page size, window and feature bits.	BUILT
UC6	Load a module while the system runs	The kernel asks for a module to be loaded into a free slot. The boot section does it on the kernel's behalf, or the kernel's own loader does, depending on the build. The kernel checks the module, records it and starts it.	MEASURED with a stand-in kernel
UC6b	Run with modules that are already in flash	A kernel built without a loader starts the modules it finds recorded. They were programmed with the firmware or loaded earlier.	MEASURED
UC7	List, stop and remove modules	Shell commands in the kernel.	PROPOSED

5.2 Requirements

ID	The loader ...	Evidence
FR-1	accepts ELF32, little-endian, executable files for the AVR machine type whose program headers start at offset 52, fed in file order. This is what avr-gcc emits.	MEASURED T1, T3, T11, host
FR-2	needs nothing after the last loadable byte. Whatever follows, including XMODEM padding, is ignored.	MEASURED T3, T4
FR-3	refuses a file built for another core family from the ELF header alone.	MEASURED T5, T7
FR-4	refuses any flash segment outside the permitted window. The loader's own pages and the image record lie outside the window.	MEASURED host
FR-5	changes nothing in flash until every program header has been accepted.	MEASURED T7
FR-6	invalidates the installed image before changing its first byte, and makes an image valid only after its last byte is written.	MEASURED T1; power cut not simulated
FR-7	has a damaged block sent again, and acknowledges a repeated block without applying it twice.	MEASURED T2
FR-8	optionally writes the EEPROM image contained in the file.	MEASURED T8
FR-9	optionally compares the file's device signature with the chip's. A mismatch prevents the commit. The signature follows the flash data in the file, so by then flash and EEPROM have changed and the device stays in the loader. The host tool makes the same check before sending.	MEASURED T9
FR-10	after a refusal sends a one-byte reason and is ready for a new transfer. If nothing was written, the old image still starts.	MEASURED T5, T6, T7
FR-11	starts an image only if its record is intact, and optionally only if the image's CRC still matches.	record MEASURED; CRC at start BUILT
FR-12	exports a service table to the running system: programming a page, and running a whole load session. Both use the caller's memory only.	MEASURED T10, M1, M3
FR-13	loads a module while the kernel runs, into the flash range the kernel names and nowhere else; the kernel image is left untouched.	MEASURED M1 to M3, M6
FR-14	lets the kernel start a recorded module again after a reset, without a loader.	MEASURED M1 to M4

ID	The loader ...	Evidence
FR-15	keeps a module built against another kernel build from running. Today the check follows the write; it should precede it.	MEASURED M5; early check PROPOSED
FR-16	works at three levels of module support chosen at compile time, and is off by default where the part is too small.	levels MEASURED M4, M8; defaults PROPOSED
FR-17	does not depend on the protoduino bootloader: on Optiboot 8 or later the kernel loads modules through `do_spm`.	MEASURED M2
FR-18	fills the boot section the fuses reserve with as many features as fit, in a set order.	MEASURED 6.9
FR-19	never has to accept a module with static RAM: the module linker script rejects it.	MEASURED

5.3 Start-up behaviour

After reset the loader probes for a sender: it transmits `C`, waits about a second, and repeats. A transfer may begin at any probe. If none begins within the configured number of probes and the image record is intact, the image is started. If the record is not intact, the loader probes for ever. In the prototype the window is always applied and comes to about five seconds with three probes MEASURED. That is too long for a product, so the entry condition is a configuration choice: always wait; wait only after an external reset; or wait only when the application asked or no valid image exists. PROPOSED

5.4 Reasons for refusal

When the loader refuses a file it cancels the transfer and sends one more byte with the reason.

Code	Name	Meaning	Flash changed?
`0x80`	FORMAT	Not a little-endian ELF32 executable for AVR with program headers at offset 52	no
`0x81`	ARCH	Built for another AVR core family	no
`0x82`	LAYOUT	No program headers, or an entry address that does not fit	no
`0x83`	RANGE	A segment falls outside the permitted window, or an offset or size does not fit the build's address width	no
`0x84`	ORDER	Segment data is not in ascending file order, or a block arrived out of sequence	no for the file; possibly for a block
`0x85`	SEGMENTS	More separate segments than the build remembers	no
`0x86`	EMPTY	Nothing to load	no
`0x87`	SINK	A sink refused a byte: today, a device signature that does not match	yes; not committed
`0x8B`	INCOMPLETE	The sender finished before the file was complete BUILT	possibly; not committed
`0x88` – `0x8A`	ABI, SLOT, RAM	Module built for another kernel; no free slot; module declares static RAM PROPOSED	no

5.5 Limits

- At most 255 program headers; at most `ELFLD_CONF_MAX_SEGMENTS` separate loadable segments after adjacent ones are merged. Plain firmware needs one.
- File offsets and sizes must fit the build's address width: 64 KB on small parts, 16 MB on large ones.
- Flash segments must arrive in ascending address order, because the page cache only moves forward. avr-gcc output does. A read-modify-write option would lift this. PROPOSED
- The serial path needs 128 bytes of RAM for an XMODEM block and one flash page more.

6 Technical design

6.1 Where the pieces live

```
src/sys/elf/elfld.h  elfld.c  elfld.conf.h     core; also builds on the host
src/sys/elf/elfsess.h                         one load session: XMODEM in, core, page cache out; caller-c
src/sys/elf/module.h  elfmod.c                module descriptor, records, and the kernel side of loading
src/cpu/avr/flash.h   flash_*.c               page writes: protoduino service table, Optiboot do_spm, dir
boot/elfboot.c  elfboot.conf.h  fit           the bootloader, with its own build and the step that fills
ld/pd-module.ld                               linker script for modules
tools/pd/                                     Node 22 ESM command-line tool: send, verify, info, abi
```

The Arduino build compiles every source file under `src/`. The bootloader has its own entry code and no C runtime, so it must stay outside that tree.

6.2 The core

```
/* Start a load. Flash bytes outside [lo, hi) are refused before anything is written. */
void    elfld_begin(elfld_t *c, elfld_addr_t lo, elfld_addr_t hi);

/* Push the next n bytes of the file. Returns ELFLD_MORE, ELFLD_DONE or an ELFLD_E_* code;
   after DONE or an error the rest of the file is ignored. */
uint8_t elfld_write(elfld_t *c, const uint8_t *p, uint8_t n);

/* Supplied by the port: store one byte. Return 0 to continue. */
uint8_t elfld_sink(uint8_t space, elfld_addr_t addr, uint8_t byte);
```

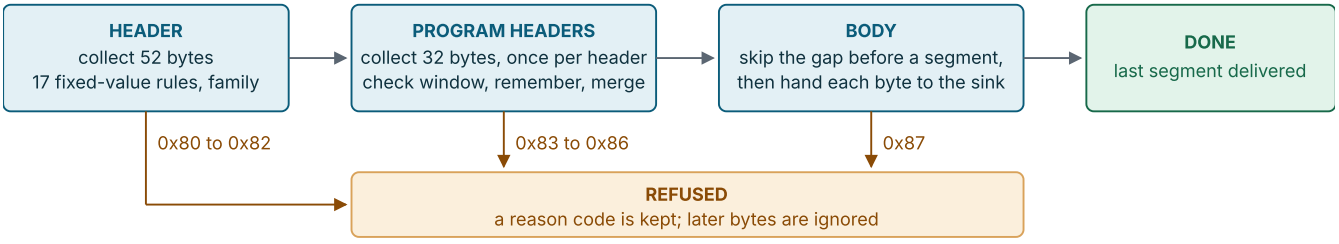


Figure 4. The core's states. Nothing reaches a sink before BODY, which is why a refusal in the first two states changes nothing.

Header rules. Seventeen bytes of the ELF header must each have one exact value: the magic, the class, the byte order, the file type, the machine, the position and size of the program headers. They are checked by a loop over a 34-byte table of offset and value pairs, which is smaller than seventeen comparisons and easy to extend.

What is remembered. The data of a segment arrives after all the headers, so each loadable segment is remembered as three numbers: the count of file bytes to skip before it, the address of its next byte, and the bytes still to deliver. Storing the gap and not the absolute offset removes the need to count file position at all. A segment that continues the previous one, both in the file and in memory, is merged into it. `.text` and `.data` always do, so plain firmware is one remembered segment.

Context layout. The scratch area for header bytes is the first member of the context, so that header bytes and the hot control fields are all within the 63-byte reach of one pointer. On AVR that matters more than it looks: see 6.10.

What building the prototype taught

- **A refused file must not cost the installed image.** The first version erased the record when the first block arrived. A wrong file then left a working board without a valid image. Invalidation now happens with the first page write, after the headers are accepted. Test T7 guards it.
- **Above 64 KB, tables in flash need the far read.** In the ATmega2560's boot section the rule table sits beyond the reach of `LPM`. The loader refused every file until the table was read through `ELPM`.
- **Service code owns no RAM.** When the application calls into the boot section, the loader's variables are the application's memory. The first service copied into the loader's page buffer and would have corrupted the caller. It now works on the caller's buffer alone.
- **A table of relative jumps does not span an 8 KB boot section.** The service table first held jumps; in the largest block they could not reach, and the link failed. It now holds addresses. Without an explicit keep, the linker's garbage collection also dropped the table and its services without a word.
- **Do not pin registers.** An attempt to force the context pointer into the Y register with a local register variable compiled, did nothing on one part and miscompiled on the other. The supported way is the compiler option in 6.10.

6.3 Flash back-end

The sink for flash is a cache of one page. A byte for a different page first commits the cached one: erase, fill the chip's page buffer, write, re-enable the read-while-write section. The cache starts as all `0xFF`, so a partly covered page is padded. The back-end is the only part that differs between chip families:

Back-end	Parts	Notes	Status
SPM from a boot section	megaAVR	Read-while-write must be re-enabled before the application section is read again	MEASURED
SPM from anywhere	classic tinyAVR	The CPU halts during a write; needs the SELFPRGEN fuse; reset vector handling in 6.6	BUILT
NVMCTRL	tinyAVR 0/1/2, AVR Dx	Different mechanism; not designed here	–

The lock bits must leave SPM writes to the application section enabled, or no bootloader of any kind can work.

6.4 Transport: XMODEM-CRC

The bootloader speaks XMODEM with 16-bit CRC. It is decades old and exactly right for this job.

Property	Why it matters here
Built into terminal programs on every platform	The unmodified `.elf` can be sent with no tool of ours installed.
The receiver sets the pace: one block, one acknowledgement	The loader programs pages while handling a block and acknowledges afterwards. No flow-control lines and no receive buffer are needed with a polled serial port.
Blocks are numbered	A block repeated after a lost acknowledgement is recognised and not applied twice.
CRC-16 per block	The same CRC routine also protects the image record and the image, so there is one checksum routine in the loader.

A block is `SOH`, the block number, its complement, 128 data bytes and two CRC bytes. The loader holds a block in RAM until its CRC has passed, and only then feeds it to the core, so damaged bytes never reach flash. One extension is used: after cancelling with two `CAN` bytes the loader sends the reason code. Terminal programs ignore it; the `pd` tool prints it.

The limits are the 128 bytes of RAM a block needs, and that XMODEM cannot tell the sender to stop early. Sending a stripped file, or using the `pd` tool, which stops after the last loadable byte, removes the wasted time. In simulation at 115200 baud a block took 13.3 ms of line time: 0.22 s for example 12 stripped, 3.06 s with its debug information. **MEASURED**

The runtime loader (cut C) uses the same framing over a protoduino pipe from the serial process. Stop-and-wait is not optional there: programming a page keeps interrupts off for several milliseconds, and a sender that kept transmitting would overrun the receiver.

6.5 Image record and memory map



ATmega328P with the 2 KB boot section, not to scale. Window 30,592 bytes; loader 1,640 to 2,036 bytes used; service table in the last twelve bytes. The tests put one 4 KB module slot at 0x6000 and its record page at 0x7000.

Figure 5. Flash layout. The record page sits directly below the loader and outside the window.

Bytes	Field	Content
0 – 1	Magic	`'p' 'D'`
2 – 3	Entry	Word address at which the image is started
4 – 6	Image end	End of the written image, rounded up to a page
7 – 8	Image CRC	CRC-16 (XMODEM polynomial) of flash from 0 to the image end
9 – 10	Record CRC	Stored so that the CRC over all eleven bytes is zero

An erased page fails both the magic and the CRC, so “no record” needs no flag. The record after test T1 read `70 44 00 00 80 01 00 13 80 c1 aa`: entry 0, image end `0x000180`, image CRC `0x8013`. The record belongs to the bootloader. A kernel's identity for modules is computed by the kernel itself (6.8), so that it also exists under Optiboot, which keeps no record.

6.6 Parts without a boot section PROPOSED

On a classic ATtiny, reset always goes to address 0, and nothing protects the loader's pages. The loader sits at the top of flash and makes four provisions.

1. **The reset vector is redirected.** When page 0 of an image passes through the page cache, the loader keeps the image's own start address for the record and replaces the first instruction with a jump to itself. The other vectors stay the image's; the loader uses no interrupts.
2. **Erase runs from the top down, page 0 last.** Before the first page is written, all application pages are erased starting just below the loader. While page 0 still holds the old redirected vector, a reset reaches the loader through it. Once page 0 is erased, everything below the loader is erased, and erased flash executes as a harmless skip instruction until the program counter runs into the loader. This closes the window urboot's documentation warns about.
3. **A landing pad.** Because that skip instruction may skip one word, the loader starts with a one-word no-op.
4. **Its own serial port.** The ATtiny85 has no USART; the build uses a bit-banged port.

The loader's own pages remain unprotected against a runaway application. On these parts the design is an updater, not a guaranteed recovery path, and an ISP programmer remains the last resort.

The ATtiny build compiles and has not executed. Points 2 and 3 were checked on the ATmega328P in simavr, which executes an erased word as the same skip instruction a real AVR does. Started in erased flash below the loader, at either word alignment and with the skip taken or not, execution arrived in the loader all four times. In a

control build without the landing pad, one of the four cases skipped the loader's first instruction and ran its rule table as code. MEASURED

6.7 The service table

The last twelve bytes of flash describe what the boot section offers to the running system.

Offset	Bytes	Content
+0	4	Address of the load-session service; 0 when the module loader is not built in
+4	4	Address of the page-write service
+8	1	Table version
+9	1	Workspace the session needs, in units of 4 bytes: 88 on the ATmega328P, which is 352 bytes
+10	2	`'p' 'D'`

```
void    write_page(uint32_t addr, const uint8_t *page);
uint8_t session(void *workspace, uint32_t lo, uint32_t hi, uint32_t *entry, uint8_t tries);

/* in the kernel: have the boot section load a module into 0x6000..0x7000 */
#define SVC (FLASHEND - 11)
if (pgm_read_byte(SVC + 10) == 'p' && pgm_read_byte(SVC + 11) == 'D') {
  uint32_t fn = pgm_read_dword(SVC);                    /* 0: no module loader in this boot section */
  uint8_t  ws[pgm_read_byte(SVC + 9) * 4];              /* the workspace the boot ROM asks for */
  uint32_t entry;
  uint8_t  result = ((session_fn)(uint16_t)(fn >> 1))(ws, 0x6000, 0x7000, &entry, 200);
}
```

Rules. A service uses the caller's memory and stack only. It switches interrupts off while it works, because the caller's interrupt handlers live in the section being programmed, and restores the caller's interrupt state on return. The session reads and writes the first serial port as the kernel left it, by polling. It blocks: the system stands still while a module is transferred, which is seconds, not milliseconds.

Why addresses and not jumps. The first version of the table held relative jumps. In an 8 KB boot section a relative jump at the end of flash cannot reach a function near the start of the section, and the link failed; and link-time relaxation is free to shorten a long jump and shift whatever follows it. Plain addresses have neither problem.

Cost and evidence. On the ATmega328P the page-write service adds 32 bytes to the loader and the session service another 364. A kernel that uses the session service is about 1.2 KB smaller than one that carries the loader itself (6.8). An application installed through the loader programmed a page through the table and read it back (T10), and a kernel had modules loaded through both services (M1, M3). MEASURED Two limits apply: simavr does not enforce the rule that SPM must run from the boot section (S2 in 8.1), so these runs show the calling convention and not the hardware permission; and on parts above 128 KB the services lie beyond a 16-bit function pointer, so the call needs the extended indirect register set. That build links and has not run.

6.8 Modules

A module is an executable linked into a flash slot. Its first bytes are a descriptor, and the ELF entry point is the descriptor's address, so the standard header field tells the loader what was installed.

Three levels of module support

The kernel is built for one of three levels. The figures are the flash size of the stand-in kernel used for the tests, messages included, on an ATmega328P.

Level	The kernel can	It carries	Flash
0. None	Nothing with modules	–	290 B

Level	The kernel can	It carries	Flash
1. Modules in flash	Start the modules that are already there, programmed with the firmware or loaded earlier. It cannot load.	Record scan, its own identity, start	656 B
2. Run time, ROM loader	Also load while running, on a protoduino bootloader built with the module loader	One call and the record write	1,148 B
2. Run time, own loader	Also load while running, on any bootloader that can write a page for it	The session and the core	2,322 B 2,414 B on Optiboot

Where it makes no sense. The level defaults to none on parts with less than 16 KB of flash or 1 KB of RAM, which covers the ATtiny25, 45 and 85, and asking for run-time loading where the workspace cannot fit is a build error. A larger part can still be built at level 0 or 1. PROPOSED The levels themselves were all run (M1 to M4, M8). MEASURED

Who writes the flash

Bootloader	How the kernel gets a page written	Status
protoduino loader	The page-write service of 6.7, or the whole session	MEASURED
Optiboot 8 or later	`do_spm`: erase, fill word by word, write	MEASURED
None; part without a boot section	The kernel executes SPM itself	PROPOSED
Any other on a megaAVR	Not possible. Level 1 still works.	MEASURED M7

In the tests the choice is made when the kernel is built. Looking at the end of flash at start-up and choosing then is straightforward and not built.

On Optiboot

A developer who prefers Optiboot keeps it. The firmware is uploaded the usual way, and the protoduino kernel loads modules itself. What Optiboot offers was read from its source (version 8.3): the second word of the bootloader is a jump to `do_spm(address, command, data)`, which performs one SPM step and re-enables the read-while-write section after an erase or a write; a build without the feature has a `ret` there instead. The kernel recognises a usable Optiboot by a major version of 8 or more in the last byte of flash and a jump in that second word. It takes Optiboot's start as 512 bytes below the end of flash, as Optiboot's own helper header does. It then programs a page as Optiboot's examples do, with interrupts off. Run against the prebuilt 8.3 image from the Optiboot repository, a module was loaded, started, recorded and started again after a reset (M2). MEASURED

Anatomy of a module

```
#include <protoduino.h>
#include <sys/process.h>
#include <sys/elf/module.h>

PROCESS_DEFINE(blink, "blink", 2,
  uint8_t  pin;                 /* all state lives in the process struct */
  uint16_t ticks;
);
PD_MODULE(blink);               /* descriptor: magic, ABI, kernel identity, &process_type_blink */

PROCESS_THREAD(blink, ev, data)
{
  blink_t *self = PROCESS_SELF(blink);
  PROCESS_BEGIN();
  /* ... ordinary protoduino process code, calling kernel functions directly ... */
  PROCESS_END();
}
```

```
# the kernel's functions, as addresses the module may link against
avr-nm firmware.elf | pd abi > kernel-abi.ld          # PROVIDE(process_post = 0x00001a2c); ...

avr-gcc -Os -mmcu=atmega328p -nostartfiles -nostdlib \
        -Wl,-T,pd-module.ld -Wl,-T,kernel-abi.ld \
        -Wl,-e,pd_module -Wl,--defsym=__pd_kernel_id=0xdeac \
        -o blink.elf blink.c
```

The source above is the intended form and is **PROPOSED**. The tests used the same recipe with a reduced descriptor and a plain function in place of the process thread **MEASURED**: an 840-byte file with one flash segment at the slot address, its entry point on the descriptor, and calls into the kernel as plain absolute `call` instructions. The module linker script turns static variables into a link error: *module has static variables: keep state in the process struct*.

Binding to the kernel

Scheme	How a module finds kernel functions	Cost on the device	Trade-off
Exact tested	Linked against the addresses of one kernel build. The descriptor carries that build's identity, and the kernel refuses a module that names another.	None. No symbol table, no names, no table of entries.	Modules must be re-linked after any kernel change. A stale module is inert, not a crash.
Table option	The kernel exposes an append-only table of jumps at a fixed address; modules link against the table slots.	Four bytes per exported function, and one extra jump per call.	Modules survive kernel updates while the table only grows.

The identity is a CRC-16 that the kernel computes over its own flash image at start-up; the host computes the same value from the kernel's ELF file and passes it to the module's link. It does not come from the bootloader's image record, because Optiboot keeps none. **MEASURED**

Slots, records and life cycle

- **Slots.** A range of flash above the kernel image and below the record pages. The tests use one slot of 4 KB. With several, the kernel names the free one before a load so the host can link for it; with 4.7, any slot.
- **Records.** A few bytes per module in a record page: a magic, the descriptor's address and the kernel identity.
- **Load.** The session writes the module's pages into the slot. The kernel then reads the descriptor, checks the magic and the identity, calls the module, and writes the record.
- **Start-up.** For every record whose descriptor still matches this kernel, start the module again. No loader is involved, which is all that level 1 needs.

- **Remove.** `process_destroy()`, which delivers the exit event, then clear the record. PROPOSED

What the tests are and are not

The kernel in M1 to M8 is a stand-in of a few hundred lines. It is not protoduino's scheduler: a module's thread is a plain function that the kernel calls three times with a block of state the kernel owns. That is enough to show a module being loaded while the kernel runs, keeping its state between calls, calling back into the kernel, being recorded, and being started from flash after a reset. Four things are still open:

- Joining it to `process_new()` and `process_start()`, and receiving through a protoduino pipe and not by polling.
- The identity check comes after the pages are written. A module built for another kernel is refused and not recorded (M5), but it has overwritten the slot by then. The session should check the descriptor while the first page is still in RAM, before any write.
- More than one slot, and removal.
- Parts above 128 KB, and hardware.

Rules and hazards

- No static variables. No interrupt handlers: the vector table belongs to the kernel.
- A process struct is at most 255 bytes, because `process_type_t.size` is one byte.
- A callback a module registers with the kernel, such as a UART receive handler, points into the module. It must be removed in the module's exit handler. The kernel cannot check this.
- Loading stops the system. With the ROM loader the kernel is not running at all during the transfer; with its own loader it could yield between blocks, but every page write still keeps interrupts off for milliseconds.
- A signature over the module, checked before the record is written, would fit in the descriptor. It is reserved, not designed.

6.9 Compile-time configuration

Configuration follows the pattern protoduino already uses in its `*.conf.h` files: every setting has a default and can be overridden. Three rules apply. Derive from the part wherever possible: address width from `FLASHEND`, expected core family from `__AVR_ARCH__`, page size from `SPM_PAGESIZE`. A feature that is off costs nothing. And fail the build, not the device: a missing setting is an `#error`, and a loader that outgrows its boot section fails at link time.

Setting	Default	Effect	Flash, 328P
`ELFLD_CONF_ADDR_BITS`	from the part	16 or 24 bits for offsets and addresses	see 6.10
`ELFLD_CONF_MAX_SEGMENTS`	3	Separate segments remembered; 6 or 9 bytes of RAM each	–
`ELFLD_FEAT_ARCH_CHECK`	on	Refuse files for another core family	12 B
`ELFLD_FEAT_COALESCE`	on	Merge adjacent segments	62 B
`ELFBOOT_FEAT_IMAGE_CRC`	on	Store the image CRC in the record	26 B
`ELFBOOT_FEAT_SERVICE`	off	Export the page-write service	+32 B
`ELFBOOT_FEAT_MODULES`	off	Export the load session too: the boot section loads modules for the kernel	+396 B
`ELFLD_FEAT_SIGNATURE`	off	Compare the device signature	+76 B
`ELFLD_FEAT_EEPROM`	off	Write the EEPROM image	+84 B
`ELFBOOT_FEAT_INFO`	off	Answer `?` with a description line	+54 B
`ELFBOOT_FEAT_VERIFY_BOOT`	off	Re-check the image CRC at every start	+62 B

Setting	Default	Effect	Flash, 328P
`ELFBOOT_VIRTUAL_RESET`, `ELFBOOT_SOFT_UART`	off	Part without a boot section (6.6); bit-banged serial	tiny only
`ELFBOOT_CONF_WINDOW`, `BAUD`	3, 115200	Probes before starting the image; line speed	–
`PD_MODULES`	from the part	Kernel: 0 none, 1 modules in flash, 2 run-time loading	see 6.8
`PD_LOADER`, `PD_FLASH`	–	Kernel: ROM or own loader; page writes through the protoduino table or Optiboot	see 6.8

Each figure is the difference from the default build of 1,640 bytes when that one setting is flipped. MEASURED The kernel settings carry the names used in the test kernel.

Filling the boot section

The fuses reserve a boot section of one of four sizes, and whatever is not used is wasted. A build step therefore starts from the smallest loader and adds features in order of priority for as long as the result still fits the chosen block. The order is itself a setting. The default puts the checks that prevent damage first and the module features next: family check, page-write service, module loader, image CRC, segment merging, signature, EEPROM, description line, CRC check at start.

Part	Block	Used	Spare	What the block then holds
ATmega328P	0.5, 1 KB	–	–	Nothing: the smallest loader is 1,540 bytes
	2 KB	2,036 B	12 B	Family check, page-write service, **module loader**, image CRC, segment merging
	4 KB	2,286 B	1,810 B	All nine features
ATmega1284P	1 KB	–	–	Nothing
	2 KB	2,022 B	26 B	Family check, page-write service
	4, 8 KB	2,834 B	1,262 B, 5,358 B	All nine features
ATmega2560	1 KB	–	–	Nothing: the smallest loader is 1,984 bytes
	2 KB	2,024 B	24 B	Family check, page-write service
	4, 8 KB	2,832 B	1,264 B, 5,360 B	All nine features

These are the results of the step as built (`fit.sh`). MEASURED Two things follow. On the ATmega328P the 2 KB block is enough for a bootloader that also loads modules for the kernel. On the large parts the 2 KB block still gives the kernel a way to write pages, so run-time loading works there with the kernel's own loader, and the 4 KB block holds everything. The space left in the larger blocks is where the next features belong: the descriptor check before the first write, a verify mode, signed images, and the fix-up stream of 4.7. On parts without a boot section the loader simply ends on a page boundary, so there is no block to fill.

6.10 Measured budgets

Build	Flash	RAM
ATmega328P, smallest	1,540 B	344 B
ATmega328P, default	1,640 B	350 B
ATmega328P, with module loader	2,036 B	352 B
ATmega328P, everything on	2,286 B	362 B
ATmega1284P, default	2,180 B	492 B
ATmega2560, smallest	1,984 B	483 B
ATmega2560, default	2,182 B	492 B

Where the 1,640 bytes go	Flash
Session: XMODEM receiver, record write, start decision	468 B
Core: delivery loop and flash sink	348 B
Core: program headers	242 B
Core: header check, rule table, begin	190 B
Page programming and commit	186 B
CRC-16, for blocks and for flash	98 B

Build	Flash	RAM
ATmega2560, everything on	2,832 B	508 B
ATtiny85, bit-banged serial BUILT	1,700 B	286 B

Where the 1,640 bytes go	Flash
Serial get and put; start-up; image start	108 B

RAM. The loader's RAM is the session workspace: on the ATmega328P the XMODEM block (128), the page buffer (128), the core context (84) and ten bytes of bookkeeping. A kernel that uses the ROM loader needs the same 352 bytes free on its stack for the duration of a load; a kernel with its own loader holds them statically.

Why it is this big, and what would shrink it. The code is ordinary C and the cost is spread evenly: ten to thirty bytes per source line, mostly 16-bit arithmetic on fields reached through a pointer. Nothing has been tried from this list, in order of expected yield:

1. Write the per-byte path and the SPM sequences in assembly, as Optiboot and urboot do.
2. Replace the 128-byte block with a smaller frame, which also removes the buffer-then-apply loop and suits the small ATtinys.
3. Build with a newer avr-gcc. The repository's `platformio.ini` already points at one; only 7.3.0 was available here.

Two things already mattered. By default avr-gcc kept the context pointer in the X register, which has no displacement addressing, and spent three instructions on each field access; with `-mstrict-X` the header check alone fell from 154 to 120 bytes. And when no service can bring a workspace of its own, binding the session to one static workspace lets the compiler address its fields directly: the default build fell from 1,756 to 1,640 bytes. MEASURED

Time. Line time dominates: 13.3 ms per block at 115200 baud in simulation, about 9.6 KB per second. Real hardware adds the time to erase and program each page, a few milliseconds each, which simavr does not model (S5 in 8.1).

7 Fixes for current protoduino caveats

These came out of reviewing the UART and serial layers as a base for the loader. Each was checked by running protoduino's real sources under protosim, except where marked as read. None has been applied. Four are defects; the rest are small.

#	Where	What happens	Evidence
1	`src/cpu/avr/uart_private_impl.h:69`	`uartN_baudrate()` reports twice the real rate whenever the double-speed bit is off. The line rate itself is correct.	**MEASURED** request 57600: registers give 58,823, function returns 117,647. Request 300: real 300, reported 600.
2	`uart_private_impl.h:81–87`	`uartN_close()` ends with an unconditional `sei()`. Called with interrupts disabled, it switches them on under the caller.	**MEASURED** interrupt flag 0 before the call, 1 after.
3	`src/sys/serial/serial_private_impl.h:293`	`serialN_flush()` tests the interrupt flag the wrong way round compared with its own documentation. With interrupts off it enables the transmit interrupt, which cannot fire, and sends nothing.	**MEASURED** ten bytes queued with interrupts off: after 60,000 calls nine are still unsent. With interrupts on all ten go out.
4	`docs/loader.md`	Describes a loader that cannot work on AVR (module code in RAM, `-fpic`) and an API that is not in `src/`. The README links to it as experimental.	**READ** avr-gcc prints "-fpic is not supported". No loader source exists in the tree.
5	`uart_private_impl.h:105–109`	The frame-error branch in `uartN_rx_error()` can never run: the flag is masked out three lines earlier. Skipping frame errors is a documented choice; the dead branch is the only defect.	**READ**
6	UART header documentation	Nothing says that the polled receive calls and the receive interrupt must not be used together. With both active the stream splits between them.	**MEASURED** input `hello`: the callback received `hlo`, the poller `el`.
7	`uart_private_impl.h`	The interrupt handlers and callback pointers are always compiled in. A bootloader that wants the same register mapping without vectors cannot have it.	**READ** The polled calls themselves work with interrupts off **MEASURED**: input `hello` echoed in full.
8	`platformio.ini`	`lib_deps` is an absolute path on one Windows machine, so the project builds nowhere else as checked in.	**READ**
9	`uart_private_impl.h:44, :162`	A baud rate of 0 divides by zero in `openex()`. A `#pragma message` prints on every build of every UART instance.	**READ**

Proposed changes

```
/* 1. uart_private_impl.h, openex(): remember which divisor is in force */
  uint8_t  samples = 8;                                  /* double speed: clock / 8 */
  uint16_t baud_setting = (F_CPU / 4 / baud - 1) / 2;
  __UCSRA__ = 1 << __U2X__;
  if (((F_CPU == 16000000UL) && (baud == 57600)) || (baud_setting > 4095)) {
    baud_setting = (F_CPU / 8 / baud - 1) / 2;
    __UCSRA__ = 0;
    samples = 16;                                        /* normal speed: clock / 16 */
  }
  ...
  CC_TMPL_VAR(baudrate) = F_CPU / ((uint32_t)samples * (baud_setting + 1));

/* 2. uart_private_impl.h, close(): leave the caller's interrupt state alone */
void CC_TMPL_FN(close)(void)
{
  CC_ATOMIC_RESTORE()
  {
    CC_TMPL_VAR(baudrate) = 0;
    __UCSRB__ &= ~(_BV(__RXEN__) | _BV(__TXEN__) | _BV(__RXCIE__) | _BV(__UDRIE__));
  }
}

/* 3. serial_private_impl.h, flush(): poll the port when interrupts are OFF */
  if (bit_is_clear(SREG, SREG_I))
  {
    if (!CC_TMPL2_FN(tx_is_ready)())
      return cnt;
    CC_TMPL2_FN(tx_write8)(ringb8_get(&VAR_TX));
    return cnt - 1;
  }
  CC_TMPL2_FN(tx_enable_int)();
  return cnt;
```

- **4.** Replace `docs/loader.md` with a short page that points here, and mark the README entry as a design, not a feature.
- **5.** Delete the dead branch. If frame errors are wanted later, report them from the receive interrupt, where the flags are read before the data register and are valid; put it behind `UART_CONF_REPORT_FRAME_ERRORS`, default off, so behaviour stays as it is.
- **6.** One paragraph in `uart_private_header.h`: use the callbacks with interrupts on, or the polled calls with interrupts off, never both.
- **7.** Add `UART_CONF_ISR`, default 1. With 0, the handlers, the callback setters and the receive-interrupt enable in `openex()` are left out, and the bootloader can use the same template.
- **8.** A relative library reference in `platformio.ini`.
- **9.** Return `ERR_SERIAL_BAUD` for a zero rate; put the message behind a verbose-build switch.

Findings from my first review that testing withdrew

They are listed so the record is straight. The polled receive calls do work after `open()` when interrupts are off. `rx_clear_errors()` does clear an overrun under simavr (S7 in 8.1); on a real chip those flags are read-only and clear when the data register is read, so the call is harmless there. The transmit path through the data-register-empty interrupt is sound: 18 bytes went out in 19 handler calls. The 57600 special case is the deliberate Arduino-compatible behaviour. The read-modify-write on the control register can at worst cause one extra interrupt that switches itself off.

8 Simulator findings: simavr and protosim

Running the prototype and the probes turned up behaviour in both simulators that matters to anyone testing firmware with them. This section lists all of it: where simavr differs from a real chip, defects in protosim, and what protosim would need to test this loader without a separate harness. The versions are protosim `6fadc6f` and simavr `a6fc4cc`, the commit protosim pins. Nothing was changed in either project. Recorded probe output is in `prototype/results-simulators.txt`.

8.1 simavr: where the simulation differs from the hardware

#	Where	What simavr does	A real AVR	Evidence
S1	`sim_elf.c:103, :255`	The firmware loader is given the last valid flash address as its limit and rejects any chunk that ends on the last byte of flash. The bytes are dropped with only a log line. This is an off-by-one.	The chunk fits.	**MEASURED** "Loading 4 bytes of Flash data at 7ffc would overflow buffer (32767 bytes)"
S2	`avr_flash.c`, SPM handling	SPM is carried out wherever the program counter is.	On parts with a boot section, SPM outside it does nothing.	**MEASURED** a program at address 0 erased and wrote a page
S3	`avr_flash.c`	A page write replaces the page with the buffer contents.	Programming only clears bits; without an erase the result is old AND new.	**MEASURED** `F0` then `3C` with no erase read back `3C`, not `30`
S4	`avr_flash.c`	There is no read-while-write state. The application section reads back correctly straight after a page write, and the busy bit is never set.	The section cannot be read until it is re-enabled.	**MEASURED** data read back at once; `SPMCSR = 04`
S5	`avr_flash.c`	Erase and write finish within the instruction. The page-write bit stays set in `SPMCSR` afterwards.	Each takes milliseconds; the control bits clear on completion.	**MEASURED** zero busy polls
S6	`sim_core.c:777`	When the program counter reaches the end of flash, the CPU is marked as crashed. Erased words themselves are executed correctly, as the skip instruction.	The program counter wraps to address 0.	**MEASURED** "CPU CRASHED" after running through erased flash with nothing above it
S7	`avr_uart.c:334–336`	Any write to the UART status register clears the overrun flag; the line that would preserve it is commented out.	The flag is read-only and clears when the data register is read.	**MEASURED** status `AA` before `rx_clear_errors()`, `A2` after
S8	`avr_uart.c`, status read	The frame-error flag is recomputed only while a received byte is waiting, and writes cannot clear it, so it stays set after the last byte has been read.	The flag belongs to the byte that is next to be read.	**READ** It matches the note in protoduino's driver that the flag cannot be cleared.
S9	`avr_uart.c`, input queue	The receiver queues 63 bytes before it reports an overrun.	Two bytes and the shift register.	**MEASURED** 63 bytes drained after 100 were sent with nobody reading

#	Where	What simavr does	A real AVR	Evidence
S10	`avr_uart.c`, receive timer	With more than one byte queued, the receive flag stays set while the interrupt is raised again only one byte time later. Code that polls with interrupts on sees the flag and can take the byte ahead of the handler.	The interrupt is taken as soon as the flag is set.	MEASURED 568 consecutive polls saw the flag at 9600 baud; none at 19200
S11	`sim_avr.c`, `avr_reset()`	A reset discards every pending cycle timer and restarts the cycle counter at zero. An external reset is not recorded in `MCUSR`.	Reset causes are all recorded.	MEASURED `MCUSR = 01` at power-on and `08` after a watchdog reset, both correct. Timers and external reset READ; effects in B6 and B7.

What this means for a bootloader tested in simulation

S2 to S5 are the ones that matter here. A loader that forgets to erase before writing, forgets to re-enable the read-while-write section, does not wait for an operation to finish, or runs outside the boot section passes every test in simavr and fails on a chip. The prototype does all four correctly, but the simulated results in section 9 are not what shows that. They establish parsing, protocol and addressing. Flash discipline, timing and power loss need hardware.

8.2 protosim: defects

#	Where	What happens	Evidence	Suggested fix
B1	`src/protosim.c:470`	The exit status is always 0: the last line is `return force_exit_success ? 0 : 0;`. A CPU crash before `--exit-on-uart` matches is reported as success. Only running out of `--max-steps` fails.	MEASURED “CPU CRASHED”, exit status 0	Non-zero on a crash, and whenever an expected string was not seen.
B2	`src/protosim.c:304`	For a bootloader given as ELF, the entry point is read from the temporary instance's program counter, which simavr never sets when it loads a file. The loader is “started” at address 0.	MEASURED `entry=0x0000` for the ELF, `0x7800` for the same loader as HEX	Use the firmware's base address or the ELF entry point.
B3	`src/protosim.c:305`	The ELF path copies the temporary instance's whole flash over the simulated flash, which wipes the application loaded just before. The HEX path overlays only the bootloader. With B2 the result looks right by accident: address 0 is now erased, and erased flash runs through to the loader.	MEASURED only the loader's banner appears; the application's never does	Copy only the bootloader's own range.
B4	`src/protosim.c:195`	A main firmware given as Intel HEX is read with simavr's single-chunk reader. Of a file with two regions, only the first is loaded.	MEASURED application and loader in one file: “Loaded HEX: 180 bytes”, the loader absent	Use simavr's multi-chunk reader, or protosim's own.

#	Where	What happens	Evidence	Suggested fix
B5	`src/protosim.c:84, :349`	`--uart0-in` injects one byte per millisecond, starting 1 ms after start, whatever the baud rate and whether or not the receiver is on. Bytes sent before the receiver is enabled are lost. At 9600 baud bytes arrive faster than a byte time and queue (S10).	MEASURED receiver enabled at 5 ms: the first 5 of 16 bytes were lost	Start when the receiver is enabled, as the TCP path does, and pace from the configured rate.
B6	`src/protosim.c:349`	Injection stops for good after any reset, because simavr's reset discards the timer that drives it (S11).	MEASURED after a watchdog reset none of the remaining bytes arrived	Register the timer again after a reset.
B7	Profiling reports	After a reset the cycle counter restarts at zero, so the report's totals no longer match the per-function sums.	MEASURED "hottest single edge: 200.0% of total runtime"; 99.4% without a reset	Keep a running total of its own.
B8	`src/protosim.c:384`	`--bootloader` always starts the STK500 uploader thread. No switch turns it off or selects another protocol. In headless mode, where no terminal device exists, the thread printed nothing after its start message.	MEASURED and READ	A switch; see 8.3.
B9	No assignment anywhere	simavr's `reset_pc` is never set, so a reset during a `--bootloader` run restarts at address 0 and not in the loader.	READ The harness set it and reset correctly.	Set it when a bootloader is given.
B10	`include/stk500_uploader.h:36`	The uploader's page size is fixed at 128 bytes, so the automatic upload suits ATmega328P-class parts only.	READ	Take it from the part.
B11	`src/ihex_loader.c:65`	The bootloader HEX reader fills gaps between regions with `00`, where erased flash is `FF`.	READ	Fill with `FF`.
B12	`--uart0-in`	The input is a C string. A file that contains zero bytes, such as an ELF, cannot be passed.	READ	See 8.3.

8.3 protosim: additions for testing this loader PROPOSED

- **Binary input.** `--uart0-in-file`, or better an XMODEM sender beside the STK500 one, with a switch for which of the two runs with `--bootloader`, and one for neither.
- **Flash comparison.** `--expect-flash <file.elf>`: at exit, compare flash with simavr's own load of that file. This is the oracle test of section 9 as a one-line check for CI, and it needs B1 fixed to be of use.
- **Reset into the loader.** B9, so that an application's request for an update can be tested.
- **A strict flash mode.** An option that reports SPM outside the boot section, a page write with no erase before it, and reads of the application section while it is busy. That would turn S2 to S4 from blind spots into test failures. It could live in simavr or as a check in protosim.

9 Verification

9.1 End to end in simulation

The loader ran inside simavr. A harness sent each file with XMODEM as a terminal program would, at 115200 baud, and afterwards compared the simulated flash below the record page with what simavr's own ELF reader produces for the same file.

Test	Scenario	Outcome
T1	328P: unmodified avr-gcc ELF, 12,860 bytes, clean line	Flash identical to the reference; record written; application reached
T2	328P: block 3 damaged in transit, acknowledgement of block 5 lost	One block sent again after a negative acknowledgement, one duplicate dropped; flash identical
T3	328P: protoduino example 12, 29,404 bytes with debug information	Flash identical; 3.06 s of line time
T4	328P: the same example stripped, 2,048 bytes	Flash identical; 0.22 s
T5	328P, empty chip: an ATmega2560 file	Refused `0x81`; flash untouched
T6	328P, empty chip: random bytes	Refused `0x80`; flash untouched
T7	328P: install, reset, then offer an ATmega2560 file	Refused `0x81`; the installed application intact and started again after the window
T8	328P, all options: file with EEPROM data and a matching signature	Flash identical; EEPROM holds `01 02 03 04 05`
T9	328P, all options: an ATmega168P file, same core family, other device	Refused `0x87` at the signature; no record written; the loader keeps control
T10	328P: an installed application programs a page through the service table	Page reads back as written; the caller's interrupt state restored
T11	2560: small image	Flash identical; record written; application reached
T12	2560: 106,830-byte image reaching far flash, 839 blocks	Flash identical; 12.0 s of line time
T13	328P: execution started in erased flash below the loader, at both word alignments, with the skip taken and not taken	The loader was reached in all four cases. In a control build without the landing pad, one case skipped the loader's first instruction and ran its rule table as code

9.2 Run-time module loading in simulation

Here the bootloader only sits in flash. A stand-in kernel runs on the ATmega328P, and the harness sends it a module with XMODEM while it runs. Afterwards the module slot is compared with simavr's own load of the module file, the kernel image is checked for changes, and the part is reset to see what starts from flash. What the kernel is, and is not, is set out in 6.8.

Test	Scenario	Outcome
M1	protoduino bootloader with the module loader; the kernel calls its session service	Slot identical to the reference; module ran and kept its state; recorded; started from flash after a reset
M2	Stock Optiboot 8.3 as the bootloader; the kernel carries the loader and writes through `do_spm`	The same
M3	protoduino bootloader with only the page-write service; the kernel carries the loader	The same
M4	Kernel built for modules in flash only; module programmed together with the firmware	Module started; the kernel has no loader

Test	Scenario	Outcome
M5	A module linked against another kernel build	Loaded into the slot, then refused by the kernel; not recorded; nothing started after a reset
M6	A firmware image offered to the module loader	Refused `0x83` before any write: it lies outside the slot
M7	Kernel built to load, on a bootloader that offers no service	The kernel reports that it cannot load and carries on
M8	Kernel built without module support, module present in flash	Module ignored

In every load the kernel image was unchanged afterwards.

9.3 On the host

The same core, compiled natively, was fed real files in chunks of random size and its output compared with `avr-objcopy`.

- Flash and EEPROM images identical for the test program built for ATmega328P, ATmega2560 and ATtiny85; identical for example 12, for its stripped form, for one-byte-at-a-time feeding, and for the 106 KB image.
- The signature bytes extracted from the three files were those of the three parts.
- Refused as designed: a file for another family (`0x81`), an image larger than the window (`0x83`), a non-ELF file, a relocatable `.o` and an x86-64 object (all `0x80`). A truncated file leaves the core waiting for more.
- A module was accepted in its own slot and refused in another; the firmware was refused in a module slot.
- The step that fills the boot section was run for every block size of three parts (6.9).

9.4 Not verified

- Anything on hardware, including fuse and lock-bit settings and real programming times.
- Loss of power during an update. The transaction argument in 4.4 is by construction.
- The ATtiny path: it has never executed. Only its run-through mechanism was checked, on the ATmega328P (T13).
- Flash discipline. simavr accepts a missing erase, a missing read-while-write re-enable and SPM from the application section (8.1), so the simulated results cannot show that the loader gets these right.
- The image CRC check at start-up, the information reply, and the service table on parts above 128 KB.
- Modules as protoduino processes: the tests use a stand-in kernel, one slot and a polled serial port (6.8).
- Optiboot on a real board, and Optiboot versions other than the 8.3 image that was used.

9.5 Test plan from here

- **Host unit tests in CI** for the core, with the files above as fixtures, and a fuzzer on `elfld_write()`. The core is small, pure C and takes untrusted input, which is the ideal fuzzing target.
- **protosim end to end** per supported part, once the additions of 8.3 exist and B1 is fixed, with the oracle comparison as the pass criterion. The module scenarios M1 to M8 belong in the same suite.
- **Hardware**: an Uno and a Mega 2560, with an ISP programmer at hand for fuses. A relay on the supply to cut power at random points of an update, hundreds of times, checking that the board always answers afterwards.

10 Roadmap, risks and open questions

Step	Deliverable	Done when
R0	The fixes of section 7	The probes in `prototype/probes` give the corrected results

Step	Deliverable	Done when
R1	Core and session in `src/sys/elf`, host tests, fuzzing	Fixtures pass in CI on the host and for two AVR parts
R2	Bootloader for ATmega328P and ATmega2560 with the fill step; protosim fixes B1 to B9 and the additions of 8.3	T1 to T13 run in protosim; first boards flashed with a terminal program
R3	Modules in the real kernel: descriptor macro, records, `process_new()` and `process_start()`; all three levels; ROM loader and own loader; Optiboot back-end	M1 to M8 pass with protoduino's scheduler, and on an Uno with Optiboot
R4	The `pd` tool; PlatformIO upload through it; `elfboot_request()`; descriptor check before the first write; several slots and removal	`pio run -t upload` sends the `.elf`; the power-cut rig passes
R5	ATtiny path on hardware; relocation by fix-up stream; signed modules; a size pass in assembly	Each on its own evidence

Risks

- **Size.** The 1 KB boot section may stay out of reach. The design does not depend on it; a 2 KB section costs the application 1 KB.
- **ATtiny recovery.** Without hardware protection for the loader's pages, a faulty application can destroy it. The claim for these parts is deliberately weaker.
- **Exact binding.** Convenient while developing, awkward for distributing modules. The table scheme is the answer and costs flash.
- **The system stops while loading.** The ROM loader blocks for the whole transfer, and any flash write keeps interrupts off for milliseconds.
- **Optiboot is a moving target.** The kernel relies on where Optiboot puts `do_spm` and on its size. Both are checked at start-up, but only version 8.3 was tried.
- **Old compiler.** All sizes are from avr-gcc 7.3.0. A newer one may be better or worse.

Open questions

- Which small part is meant by “ATtiny8”? The answer decides whether the small-frame transport is needed early.
- Should updates be authenticated, and if so with what? A keyed hash costs several hundred bytes.
- Which binding scheme is the default for modules: exact or table?
- May loading a module stop the system for seconds, as the ROM loader does, or must it be cooperative?
- Is the default order for filling the boot section the right one?
- Should the loader wait after every reset, or only on request? It changes start-up time by seconds.
- Where should the bootloader's build live, given that it cannot be under `src/`?

A ELF fields the loader reads

Where	Offset	Field	Use
ELF header	0 – 3	magic	Must be `7F 45 4C 46`
	4, 5	class, data	Must be 1 and 1: 32-bit, little-endian
	16	`e_type`	Must be 2, an executable. A relocatable `.o` is refused here.
	18	`e_machine`	Must be 83, AVR
	24	`e_entry`	Start address; for a module, the address of its descriptor
	28	`e_phoff`	Must be 52: program headers directly after the ELF header

Where	Offset	Field	Use
	36	`e_flags`	Low 7 bits name the core family; compared with `__AVR_ARCH__`
	42, 44	`e_phentsize`, `e_phnum`	Must be 32; count of program headers, below 256
Program header	0	`p_type`	Only 1, a loadable segment, is considered
	4	`p_offset`	Where the segment's bytes are in the file; must not go backwards
	8, 20	`p_vaddr`, `p_memsz`	Run address and size in memory; used only to record a RAM extent
	12	`p_paddr`	Load address. Its top byte selects the memory; the rest is the address there.
	16	`p_filesz`	Bytes to deliver; a segment with none is skipped

For the ATmega328P test program avr-gcc wrote six loadable segments: code at load address 0; initial `.data` at run address `0x800100` but load address `0x136`, directly after the code; `.bss` with no file bytes; EEPROM at `0x810000`; fuses at `0x820000`; signature at `0x840000`. **MEASURED**

B Reproducing the measurements

Everything is in `docs/elfloader/prototype/`. It needs `gcc-avr`, `avr-libc`, `libelf-dev` and protosim built with its simavr submodule. The Optiboot image is not included: copy `optiboot_atmega328.hex` from the Optiboot repository into `fixtures/` for M2, M4 and M8.

```
./make_fixtures.sh                      # test files, and the stand-in kernels with their modules

# core on the host, against a file built with avr-gcc
gcc -std=c99 -O2 -DELFLD_FEAT_EEPROM=1 -DELFLD_FEAT_SIGNATURE=1 -o host_test host_test.c elfld.c
./host_test fixtures/app-atmega328p.elf 5 out.bin

# the harness that runs the loader inside simavr
S=../../../../protosim/libraries/simavr/simavr
gcc -O2 -o sim_harness sim_harness.c -I$S/sim -I$S/sim/avr \
    $S/obj-x86_64-linux-gnu/libsimavr.a -lelf -lm -lpthread

./run_tests.sh                          # T1 to T13
./run_modules.sh                        # M1 to M8
./fit.sh atmega328p 2048                # fill one boot section
./build.sh m2560 atmega2560 0x3F000 -DELFBOOT_FEAT_MODULES=1
```

Recorded outputs are in `results-sizes.txt`, `results-fit.txt`, `results-tests.txt`, `results-modules.txt` and `results-simulators.txt`. The UART and serial findings of section 7 come from `probes/`, built against protoduino's own sources and run with `protosim --uart0-in … --uart0-out …`; the simulator findings of section 8 from `simprobes/` and `sim_harness --fallthrough`.

Sources

- Contiki, `core/loader/elfloader.c`, `symtab.c`, `cle.c`: github.com/contiki-os/contiki. Removal: github.com/contiki-ng/contiki-ng, commit 6220aea14.
- simple-avr-bootloader: github.com/m3y54m/simple-avr-bootloader
- Optiboot: github.com/Optiboot/optiboot, read at commit e363ecb: `optiboot.c` (version 8.3, `do_spm`, `pre_main`) and `examples/demo_dospm/optiboot.h`. Version 8 notes: forum.arduino.cc/t/optiboot-now-at-version-8-0/547763
- urboot, on erased flash and the first-page hazard: github.com/stefanrueger/urboot. Optiboot on ATtiny: github.com/SpenceKonde/ATTinyCore, `avr/extras/Ref_Optiboot.md`
- micronucleus and its reset-vector patch: github.com/micronucleus/micronucleus
- Zephyr LLEXT and symbol link identifiers: docs.zephyrproject.org/3.7.0/services/llext/api.html

- Microchip, ATmega48A/88A/168A/328P family, “Self-Programming the Flash” and “USART Control and Status Register 0 A”: onlinedocs.microchip.com
- avr-libc memory sections: the avr-libc user manual, “Memory Sections”
- protoduino and protosim sources at the branch named on the cover; simavr at commit a6fc4cc.